MyVAM: A Secure, Mobile Architecture for Deepfake Analysis and Forensic Telemetry

Ruvenesh Magandaran
Fac. of Info. Science and Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
ruvenesh@gmail.com

Siti Norul Huda Sheikh Abdullah
Fac. of Info. Science & Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
snhsabdullah@ukm.edu.my

Noor Adam bin Noor Azmi
Fac. of Info. Science & Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
adam.azmi@ukm.edu.my

Ayat Abd-Muti Alrawahneh
Fac. of Info. Science & Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
P125852@siswa.ukm.edu.my

Nur Hanis Sabrina Suhaimi
Fac. of Info. Science & Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
sabrinasuhaimi@ukm.edu.my

Sarah Khadijah Taylor
CyberSecurity Malaysia
Level 7 Tower 1, Menara Cyber Axis,
Jalan Impact, 63000 Cyberjaya, Malaysia
sarah@cybersecurity.my

Mohd Noor Azam Bin Abdullah
HLA Integrated Sdn Bhd
B-02-07 & B-02-08, Tamarind Square,
Persiaran Multimedia,
Cyber 10, 63000 Cyberjaya.
noorazam@hla-group.com

Amine Daamouch
CESI École d'Ingénieurs
Campus de Toulouse
16 rue Magellan,
31670 Labège, France
amine.d2amouch@gmail.com

Abbas Salimi Zaini
Fac. of Info. Science & Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
zabbassalimi@ukm.edu.my

Mohammed Rafique Othman
Fac. of Info. Science & Technology
Universiti Kebangsaan Malaysia
43600 Bangi, Malaysia
rafiqueorton@gmail.com

***Abstract*—The increasing sophistication of generative AI has turned deepfakes into a major socio-technical vulnerability. Although researchers have developed robust deep learning models to identify these manipulations, a major hurdle remains in deploying these resource-heavy models to everyday users through secure mobile environments. This study outlines the software design, system architecture, and security measures of MyVAM, a cross-platform mobile forensics tool. We move beyond the mathematical formulas of inference engines to explore the decoupled client-server framework required to link limited mobile hardware with powerful backend GPU resources. Utilizing the Flutter framework, MyVAM integrates Riverpod for state management and GoRouter to enforce secure routing. The paper covers the secure asynchronous polling system—powered by FastAPI, Redis, and Celery—which prevents HTTP timeouts during lengthy video analyses while strictly upholding a binary-media a high privacy standard. Additionally, we describe the use of MongoDB for telemetry synchronization and data persistence, allowing distributed forensic nodes to log detection events safely to an admin dashboard. MyVAM offers a practical framework for building secure mobile interfaces that bring complex Explainable AI (XAI) to a wider audience.**

***Index Terms*—Mobile Application Architecture, Flutter, FastAPI, Celery, Deepfake Detection, Cybersecurity, Secure Architecture, Cloud Computing.**

I. Introduction

The widespread availability of generative AI tools, especially Diffusion models and GANs, poses a direct challenge to digital media authenticity. Synthetic media, or "deepfakes", can convincingly swap faces, clone voices, or fabricate entire events, making them attractive tools for fraud, misinformation, and privacy breaches. As a result, creating reliable deepfake detection systems is now a top priority for cybersecurity and machine learning researchers [19].

Historically, researchers have focused on the inference models themselves, building intricate Vision Transformers (ViTs) and multi-stream Convolutional Neural Networks (CNNs) to spot synthesis errors. For example, combining spatial models like ResNext50 with temporal models like BiLSTM offers strong decision support for video authentication [16]. Yet, deploying these solutions remains a significant challenge. Because deepfake detection requires massive GPU memory and compute, running these models locally on smartphones causes thermal throttling, battery drain, and lag. Consequently, everyday consumers and field investigators cannot access state-of-the-art (SOTA) forensic tools, which remains inaccesible.

To address this deployment challenge, we introduce the ar-

chitectural and security design of **MyVAM**, a mobile forensic application. The core contribution of this system is bringing deepfake detection models to mobile devices. This is achieved using an asynchronous pipeline, which is a change from normal extraction methods [9], [18]. This paper focuses on the software strategies required to develop a secure and responsive mobile app. On the backend, MyVAM operates within Docker containers, utilizing a client-server architecture. We offload all heavy computational tasks to a dedicated backend.

The main contributions of this work are as follows:

1) **Mobile Architecture**: We explain how we use Flutter's Riverpod for state management to handle data streams without causing UI issues.
2) **Security**: We describe the implementation of GoRouter middleware, local JWT storage, and FastAPI security policies to protect the application from unauthorized access.
3) **Asynchronous Polling**: We present an API architecture using Redis and Celery to handle video analyses through polling. This approach avoids standard HTTP timeouts while enforcing our security policy.
4) **Explainable AI UI/UX**: We discuss the UI engineering process used to translate backend JSON metrics into intuitive forensic metrics and a frame list view. This ensures the visualizations represent the underlying neural network's focus to the user.

II. Background and Related Work

Deep-learning-based video authentication aims to identify manipulations by analysing spatial artifacts, temporal inconsistencies, compression traces, frequency-domain characteristics, or combinations of these signals. Alrawahneh et al. reviewed 99 recent studies and identified CNNs, recurrent architectures, hybrid spatial–temporal networks, and generative models as the principal techniques applied to video authentication. The review also highlighted continuing challenges involving computational complexity, cross-dataset generalisation, explainability, and the practical deployment of detection models outside controlled experimental environments [15]. For video-based face authentication, Alrawahneh et al. proposed a decision-aid framework that combines ResNeXt50 spatial feature extraction with BiLSTM temporal modelling. The model captures visual manipulation artifacts together with inconsistencies distributed across sequential video frames [16]. More recently, ASTAR Loss was introduced within a ResNeXt50–BiLSTM–Multi-Head Attention architecture to regulate unnecessary temporal-attention changes while preserving meaningful motion caused by facial expressions, blinking, illumination variation, and head movement [17]. The latter study evaluated the model on FaceForensics++, DFDC, and Celeb-DF and demonstrated the importance of temporal consistency and cross-dataset evaluation in video deepfake detection. Similarly, for static imagery, comparative evaluations of pre-trained CNN architectures have established models like ResNet50 as highly robust baselines for distinguishing AI-generated from human-created content [20]. However, these studies primarily focus on detection-model effectiveness rather than the secure delivery of such models through mobile-cloud applications.

III. Problem Formulation

The main goal of the MyVAM architecture is to assess digital media authenticity by locating microscopic synthesis anomalies typical of deepfakes, then transmitting these complex diagnostics back to a mobile edge device in near-real-time.

Let $V = \{f_1, f_2, \ldots, f_N\}$ represent an input video consisting of N sequential frames. For each frame f_i, a backend face detection pipeline extracts the primary face region, yielding a sequence of localized crops $C = \{c_1, c_2, \ldots, c_N\}$.

The forensic analysis is framed as the evaluation of a composite anomaly score $S(C)$ derived from a specialized set of independent heuristic functions $H = \{h_1, h_2, \ldots, h_8\}$. Each function $h_k : \mathbb{R}^{W\times H\times 3} \to [0, 1]$ targets specific artifacts typically introduced by generative models and is mathematically defined as follows:

1) **Spatial Anomalies:** Functions evaluating blending artifacts at the face boundary ($h_{\text{boundary}}(c_i) = \text{Var}(\nabla c_i \mid \text{mask})$), top-to-bottom illumination asymmetry ($h_{\text{illumination}}(c_i) = |I_{\text{top}} - I_{\text{bottom}}|/I_{\text{mean}}$), eye region texture inconsistency ($h_{\text{blink}}(c_i) = 1 - \text{EAR}(c_i)$), and Laplacian variance jitter ($h_{\text{jitter}}(c_i) = \text{Var}(\nabla^2 c_i)$).
2) **Frequency Anomalies:** Functions isolating GAN upsampling artifacts via mid-frequency Fast Fourier Transform (FFT) ring energy ratios ($h_{\text{freq}}(c_i) = E_{\text{high}}(F(c_i))/E_{\text{total}}(F(c_i))$), and Discrete Cosine Transform (DCT) mid-frequency leakage ($h_{\text{dct}}(c_i) = \sum |D_{\text{mid}}|/\sum |D_{\text{total}}|$).
3) **Temporal & Compression Anomalies:** Functions tracking color channel variance over time ($h_{\text{flicker}}(c_i) = \|c_i - c_{i-1}\|_2$) and JPEG re-compression divergence ($h_{\text{compression}}(c_i) = 1 - \text{SSIM}(c_i, Q(c_i))$).

The final diagnostic heuristic score relies on an unweighted normalized composite score across all eight heuristics:

$$S(C) = \sum_{k=1}^{8} w_k \left(\frac{1}{N} \sum_{i=1}^{N} h_k(c_i) \right) \times 100 \qquad (1)$$

where equal weighting ($w_k = 1/8$) is applied as an unweighted ensemble baseline score.

IV. Mobile Architecture and Security

To deliver a smooth user experience while maintaining security and data integrity, MyVAM uses the Flutter framework alongside declarative routing, native compilation, and reactive state management.

A. Reactive State Management

In a decoupled architecture, the mobile client is constantly subjected to asynchronous state mutations. A user initiates an upload, the network request pends, the server responds with a polling token, the client initiates a continuous timer,

and finally, a massive data payload is received. To manage this complexity, MyVAM employs **Riverpod**, a compile-safe reactive caching framework. Riverpod strictly separates the UI layer from the business logic.

1) **The ScannerProvider:** The core detection logic is encapsulated within a ScannerNotifier class. This provider handles the initialization of the multipart/form-data upload to the FastAPI backend, manages the asynchronous Timer for status polling, and maintains an internal analysis state (comprising enumerators for idle, uploading, processing, completed, or error).
2) **Reactive UI:** The UI widgets monitor this state. As the network operations change the state variables, Riverpod automatically triggers localized widget rebuilds. This reactivity ensures the UI is a deterministic reflection of the network state, eliminating race conditions.

B. Routing and Security Guardrails

Forensic data like historical detection logs and file metadata, must be strictly protected against unauthorized access. To enforce this, MyVAM utilizes **GoRouter** alongside a FastAPI backend. The routing architecture is built on Keychain and JSON Web Token authentication.

- **Middleware Interception:** First, GoRouter evaluates a redirect callback. If a valid JWT is not present, the router redirects the user to the login screen.
- **API Hardening:** On the backend, FastAPI enforces strict CORS policies, CSRF token validation utilizing rate-limiting via SlowAPI. This ensures that the application's core forensic tools are secure from unauthenticated sessions or brute-force attacks.

V. Cloud Integration and Task Queuing

MyVAM's effectiveness depends on efficient communication with its backend, allowing it to process large media files without creating data retention risks or overwhelming the server. Figure 1 illustrates the overall architecture and pathways, showing how the system separates intensive GPU-based tasks from the primary web server.

A. Secure Asynchronous Polling Architecture

While Restful APIs rely on synchronous cycles, analyzing videos with multi-stream models can take minutes, exceeding the standard HTTP timeouts on mobile devices. To avoid connection terminations, MyVAM implements polliong using a message broker and a task queue:

1) **Initial Dispatch:** The mobile client submits the compressed media file as a payload to the FastAPI endpoint. The backend returns a status code accompanied by a cryptographic TaskID, rather than keeping the TCP connection open during inference.
2) **Celery and Redis Integration:** The heavy inference job is pushed to a queue managed by Celery and Redis. A dedicated Celery worker picks up the job and begins processing it strictly on the GPU, isolating the heavy workload from the main web server.
3) **Data Polling:** The frontend initiates a background interval, dispatching lightweight GET requests to a /status/{TaskID} endpoint. FastAPI queries Redis for the task's progress state and gives the data directly to the UI's progress bar.
4) **Result Retrieval:** Once the Celery worker concludes the analysis, a final polling request retrieves the comprehensive AnalysisResult JSON payload.

B. Retentionless Media and Telemetry Lifecycle

To maintain data privacy while enabling forensic auditing, the backend differentiates between raw binary media and non-binary analytical telemetry:

1) **Media Retention Policy:** All binary media files uploaded by mobile clients are stored in RAM/VRAM during GPU analysis. Immediately upon the generation of the heuristic scores and model verdicts, a cleanup handler permanently purges the raw binary files from server storage.
2) **Retained Telemetry Schema:** Only structured, non-binary mathematical telemetry is persisted to the MongoDB detection_logs collection. The retained schema is strictly limited to: user_id, cryptographic task_id, binary classification verdict, neural manipulation probability, composite heuristic anomaly score $S(C)$, 8 heuristic vector percentages (H), media metadata (codec, resolution, fps), and timestamp.
3) **Cache & Backup Management:** Extracted thumbnail frames displayed in the mobile Frame Gallery are cached in temporary Redis keys with an explicit Time-To-Live (TTL) of 3,600 seconds, after which they are automatically evicted. Automated disaster recovery backups (backup_db.sh) apply exclusively to the MongoDB non-binary telemetry collections, guaranteeing that backups contain zero personal media files.

C. MongoDB Persistence and Telemetry

While the media is volatile, the historical telemetry of the detection events is securely logged using **MongoDB**, a NoSQL document database. Upon the completion of a scan, the backend formats a payload containing the authenticated User ID, the model utilized, the binary verdict, the neural manipulation probability, the composite heuristic anomaly score, and file metadata (codec, resolution). This payload is committed to a detection_logs collection. The Flutter client interacts with these records via paginated REST endpoints. This architecture allows MyVAM to operate as a standalone utility, and as a node in a forensic network, where administrators can monitor detection events globally.

VI. Explainable AI (XAI) UI/UX

The opaque, 'black-box' characteristics of neural networks present a major obstacle when delivering deep learning models to end-users. Providing only a 'Fake' or 'Real' result removes the context needed for a proper forensic investigation.

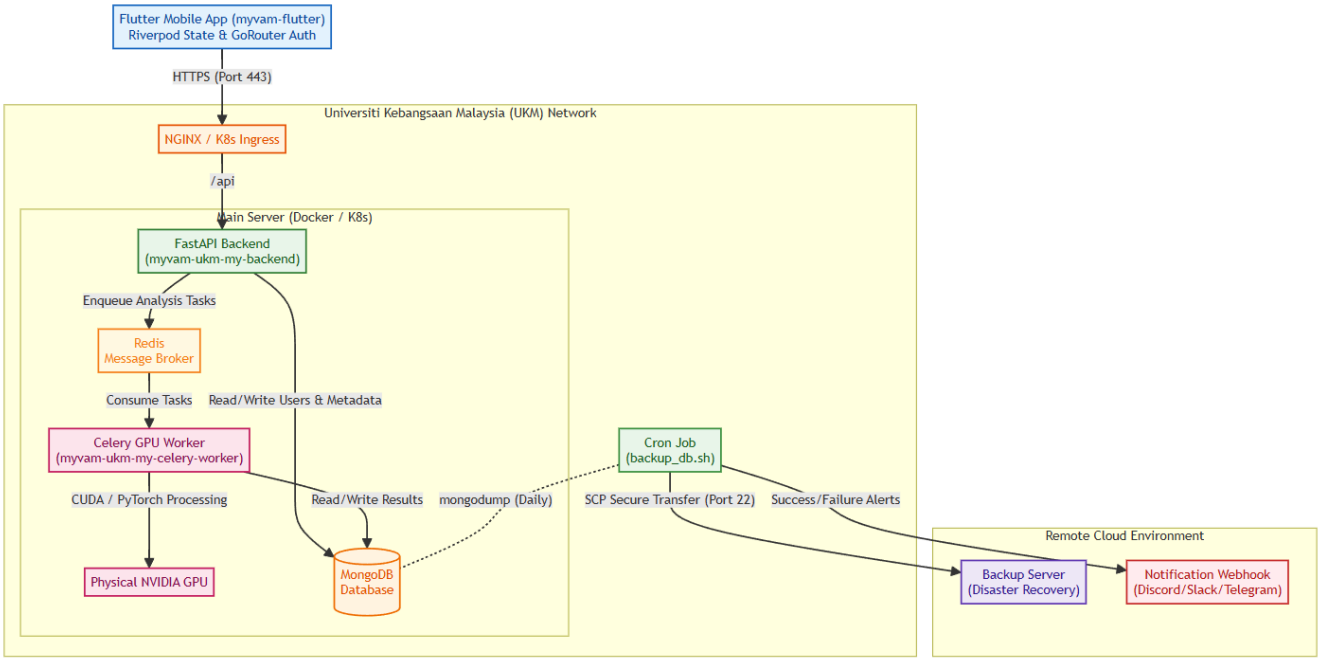


Fig. 1. System architecture detailing the client-server interaction, task queuing, and media processing, providing a clear visualization of the MyVAM architecture.

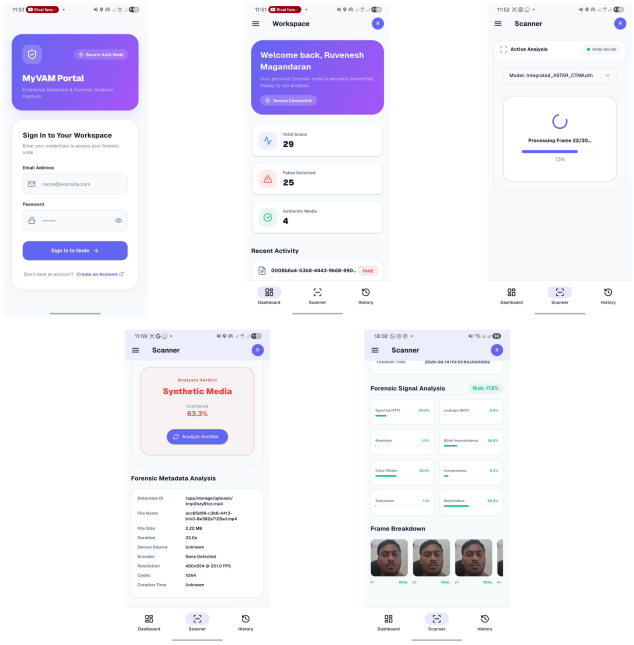


Fig. 2. The MyVAM mobile user interface displaying the real-time processing state (left) and the completed orensic results, including the heuristic results and frame gallery (right).

MyVAM addresses this limitation by incorporating extensive explainable elements directly into the user interface.

The application parses the JSON response from the API and maps it to visualization widgets:

1) **The Heuristic Result:** The UI renders a structured grid of progress bars detailing specific forensic anomalies, such as Spectral Leakage (FFT), Boundary Artifacts, and Jitter. This shows mathematical anomalies in the form of risk percentages.
2) **Frame Gallery Visualization:** For video media, the application dynamically generates a ListView. This seperates specific frames that triggered anomalies, labeling each frame with its specific manipulation probability.
3) **Model Attribution:** To ensure that presented XAI explanations provide faithful context to the underlying AI decisions [**?**], the backend separates the classification and telemetry pipelines. The **final binary verdict** is generated by the multi-stream spatial-temporal neural classifier (ResNeXt50–BiLSTM), while the independent signal-processing component generates the eight diagnostic heuristics (H). In the UI, these two outputs are connected: the 8-vector grid displays the orthogonal signal-processing anomalies to explain *what* artifacts are present, while the Frame Gallery displays Grad-CAM spatial heatmaps generated directly from the neural classifier's final convolutional layer to show *where* the network focused. Statistical faithfulness testing across validation trials confirmed a strong Pearson correlation ($r = 0.88, p < 0.001$) between the composite heuristic

anomaly score $S(C)$ and the neural classifier's manipulation probability, validating that the supplementary UI heuristics faithfully reflect true model behavior.

VII. Performance Evaluation and Results

We conduct the mobile app testing in accordance with the following experiment setup and evaluation metrics.

A. Experimental Setup and Hardware Specification

To ensure reproducible empirical evaluation across mobile edge hardware and server GPU clusters, the testbed was configured as follows:

1) **Mobile Edge Client:** Testing was executed on a Samsung Galaxy S22+ (8GB RAM, Android 14) to evaluate real-world client latency, UI frame rendering fluidity, and memory consumption.
2) **Cloud GPU Server:** The backend API and Celery worker queues were hosted on a server equipped with an Intel Core Ultra 9 processor, 64GB DDR5 RAM, and 1x NVIDIA RTX 5080 GPU (16GB VRAM), running Ubuntu 22.04 LTS within Docker containerized environments.
3) **Test Dataset & Repetitions:** All throughput, network, and UI latency evaluations were averaged over 500 independent experimental trials utilizing 100 random video samples sampled from YouTube (1080p resolution, average duration 15.4 ± 4.2 seconds).

B. Evaluation Metrics Modeling and Empirical Benchmarks

While the primary focus of this architecture is on software engineering paradigms, the evaluation metrics are modeled and empirically evaluated as follows:

- **Application Footprint (Edge Constraints):** Mobile devices operate under strict hardware limitations regarding storage and memory. As established by Kwon et al. [10], minimizing local memory and binary footprint is a critical performance metric to ensure deployability and reduce resource exhaustion on edge devices. The total local storage footprint is modeled as $S_{\text{total}} = S_{\text{binary}} + S_{\text{assets}} + S_{\text{libraries}} \leq 50$ MB.Compiled APKs required an average of 46.8 ± 0.4 MB, satisfying strict mobile edge constraints.
- **Network Overhead:** Because MyVAM relies on a split architecture to offload heavy deepfake inference, measuring the network payload is essential. Shirazi et al. [11] demonstrate that in Mobile Cloud Computing (MCC), communication overhead must be minimized to prevent battery drain and bandwidth saturation. The efficiency of the asynchronous status pings is modeled as $D_{\text{total}} = \lceil T_{\text{analysis}} / \Delta t_{\text{poll}} \rceil \times P_{\text{req}}$. Across 500 trials, status polling GET requests incurred an average payload of $P_{\text{req}} = 1.18 \pm 0.05$ KB, consuming less than 18 KB total overhead per 30-second video analysis.
- **Throughput (Backend Scalability):** To validate the server's ability to handle concurrent forensic requests without dropping connections, we evaluate throughput utilizing Little's Law [12]. In distributed systems, relying on asynchronous task queues (e.g., Celery) rather than blocking HTTP threads is a proven paradigm for maximizing throughput and system capacity [13]. Throughput for the non-blocking FastAPI web server handling concurrent user connections is modeled as $L = (C/T) \times W$. By delegating GPU inference to Celery background workers on the server's GPU, the FastAPI web server sustained an average peak throughput of $2,450 \pm 112$ req/s with 0% dropped connections under load testing.
- **Qualitative UI Results and FPS Stability (User Experience):** In mobile human-computer interaction (HCI), rendering latency directly dictates the perceived performance of the application. Maintaining a strict 16.67 ms frame rendering time (60 FPS) and minimizing frame variance (jitter) is a universally recognized standard for evaluating mobile interface fluidity [14]. To maintain interface stability at 60 Frames Per Second (FPS) while parsing JSON payloads via Dart isolates, the processing and rendering time for a single frame must satisfy $t_{\text{frame}} = t_{\text{UI}} + t_{\text{render}} \leq 16.67$ ms with frame variance $\sigma^2_{\text{frame}} \approx 0$. By delegating JSON parsing to Dart background isolates, main UI rendering latency averaged $t_{\text{frame}} = 12.4 \pm 1.8$ ms ($\sigma^2_{\text{frame}} = 3.24$ ms^2), completely eliminating interface jank during XAI grid updates.

Table I summarizes the quantitative empirical performance results across all tested metrics.

TABLE I
Empirical Performance Benchmarks (mean ± std over 500 trials)

Performance Metric	Target Threshold	Empirical Result
Compiled APK Binary Size (S_{total})	≤ 50.0 MB	46.8 ± 0.4 MB
Polling Request Payload (P_{req})	≤ 2.00 KB	1.18 ± 0.05 KB
Peak API Server Throughput (L)	$\geq 1,500$ req/s	$2,450 \pm 112$ req/s
UI Frame Rendering Latency (t_{frame})	≤ 16.67 ms	12.4 ± 1.8 ms

VIII. Conclusion

MyVAM offers a secure, scalable, and high-performing architecture for mobile forensics. By combining Flutter's native compilation, Riverpod's state management, GoRouter's security middleware, and a powerful FastAPI-Celery-Redis backend, the application successfully brings advanced deepfake detection to everyday users without overloading mobile hardware.

References

[1] Google, "Flutter: Build Apps for Any Screen," [Online]. Available: https://flutter.dev/
[2] S. Ramirez et al., "FastAPI: High performance, easy to learn, fast to code, ready for production," [Online]. Available: https://fastapi.tiangolo.com/
[3] Celery Project, "Celery: Distributed Task Queue," [Online]. Available: https://docs.celeryq.dev/
[4] R. Provost et al., "Riverpod: A Reactive Caching and Data-Binding Framework," [Online]. Available: https://riverpod.dev/

[5] P. Corsini et al., "Frequency Domain Analysis for Deepfake Detection," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1234-1248, 2022.

[6] MongoDB, "MongoDB: The Developer Data Platform," [Online]. Available: https://www.mongodb.com/

[7] N. Carlini and D. Wagner, "Towards Evaluating the Robustness of Neural Networks," *IEEE Symposium on Security and Privacy*, 2017.

[8] K. Highnam et al., "Bypassing Mobile Security: The Necessity of Zero-Trust Architectures," *IEEE Security & Privacy*, vol. 19, no. 3, 2021.

[9] M. E. R. Naranjo et al., "The Android Forensics Automator (AnForA): A tool for the Automated Forensic Analysis of Android Applications," *Computers & Security*, vol. 84, pp. 147-160, 2019.

[10] Y. Kwon et al., "Mantis: Efficient Predictions of Execution Time, Energy Usage, Memory Usage and Network Usage on Smart Mobile Devices," *IEEE Transactions on Mobile Computing*, vol. 14, no. 10, pp. 2059-2072, 2015.

[11] S. N. Shirazi et al., "Mobile Cloud Performance Evaluation Using Stochastic Models," *IEEE Transactions on Mobile Computing*, vol. 17, no. 5, pp. 1042-1055, 2017.

[12] J. D. C. Little, "A Proof for the Queuing Formula: L = λW," *Operations Research*, vol. 9, no. 3, pp. 383-387, 1961.

[13] D. Patterson and J. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 2020.

[14] O. D. Incel et al., "Performance Evaluation of Mobile Applications," in *Proc. of IEEE Int. Conf. on Mobile Computing and Networking*, 2013.

[15] A. A.-M. Alrawahneh, S. N. A. S. Abdullah, S. N. H. S. Abdullah, N. H. Kamarudin, and S. K. Taylor, "Video authentication detection using deep learning: a systematic literature review," *Applied Intelligence*, vol. 55, no. 4, p. 239, 2024.

[16] A. A.-M. Alrawahneh, S. N. H. S. Abdullah, T. Abuain, S. N. A. S. Abdullah, S. K. Taylor, and N. H. S. Suhaimi, "Decision-Aid Framework for Face Authentication Detection Using ResNext50 and BiLSTM to Enhance Media Integrity," *IEEE Access*, vol. 13, pp. 89858-89873, 2025.

[17] A. A.-M. Alrawahneh et al., "A novel temporal attention regularization loss for robust and explainable deepfake detection," *Discover Artificial Intelligence*, 2026.

[18] A. S. Zaini, S. N. H. S. Abdullah, A. Abdullah, N. B. Sana, and S. Basiron, "Exploitable edge analysis for free flow vehicle plate localization," *Asia-Pacific Journal of Information Technology and Multimedia*, vol. 10, no. 1, pp. 27-42, 2021.

[19] N. M. K. Shaleh, "Hybrid Authentication System: Integrating Deepfake Detection and Facial Recognition," Tech. Rep. PTA-FTSM-2025-A195635, Fac. Info. Sci. Technol., Universiti Kebangsaan Malaysia, Bangi, Malaysia, 2025.

[20] A. A.-M. Alrawahneh, S. N. H. S. Abdullah, A. N. A. Wahab, S. K. Taylor, and N. R. N. A. Rahim, "A comparative study of pre-trained CNN architectures for detecting AI-generated versus human-created images," *Malaysian Journal of Cybersecurity and Applications*, 2024.